Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency

Sverrir Thorgeirsson*
AST Lab
ETH Zurich
Zurich, Switzerland
sverrir.thorgeirsson@inf.ethz.ch

Theo B. Weidmann*
ETH Zurich
Zurich, Switzerland
theo@weidmann.cool

Zhendong Su
ETH Zurich
Zurich, Switzerland
zhendong.su@inf.ethz.ch

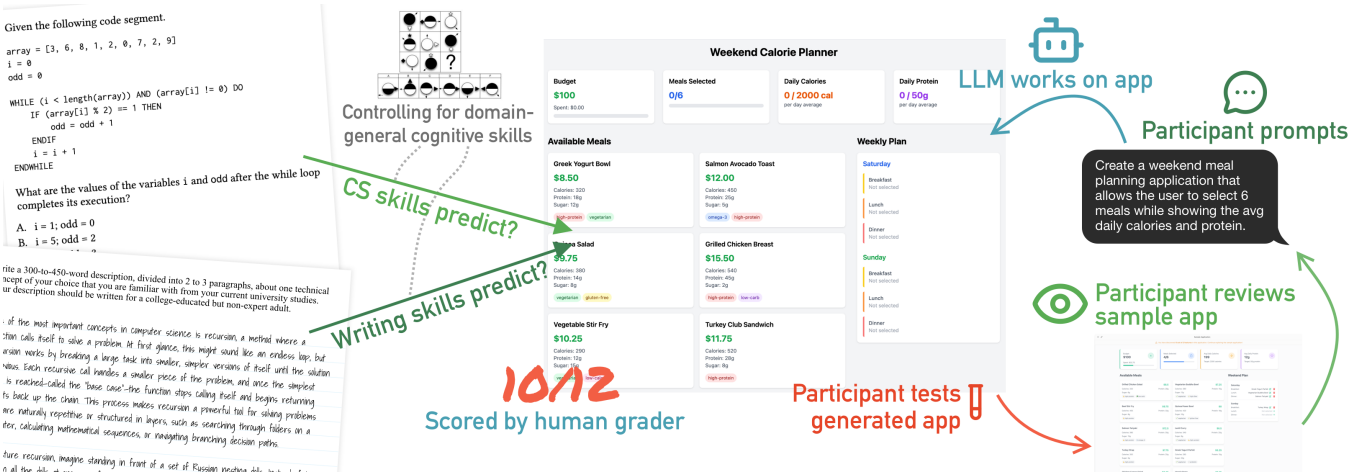


Figure 1: Our study investigates how writing skills and CS achievement relate to the ability to vibe code when controlling for domain-general cognitive skills. Participants reviewed sample applications, drafted prompts to an LLM-based agent, tested the resulting applications, and made further refinements. The final applications were then scored by a human grader.

Abstract

Many software development platforms now support LLM-driven programming, or "vibe coding", a technique that allows one to specify programs in natural language and iterate from observed behavior, all without directly editing source code. While its adoption is accelerating, little is known about which skills best predict success in this workflow. We report a preregistered cross-sectional study with tertiary-level students (N = 100) who completed measures of computer-science achievement, domain-general cognitive skills, written-communication proficiency, and a vibe-coding assessment. Tasks were curated via an eight-expert consensus process and executed in a purpose-built, vibe-coding environment that mirrors commercial tools while enabling controlled evaluation. We find that both writing skill and CS achievement are significant predictors of vibe-coding performance, and that CS achievement remains a significant predictor after controlling for domain-general cognitive skills. The results may inform tool and curriculum design, including when to emphasize prompt-writing versus CS fundamentals to support future software creators.

CCS Concepts

• **Human-centered computing → Empirical studies in HCI**.

Keywords

education, learning, vibe coding, empirical study, lab study

ACM Reference Format:
Sverrir Thorgeirsson, Theo B. Weidmann, and Zhendong Su. 2026. Computer Science Achievement and Writing Skills Predict Vibe Coding Proficiency. In *Proceedings of the 2026 CHI Conference on Human Factors in Computing Systems (CHI '26), April 13–17, 2026, Barcelona, Spain.* ACM, New York, NY, USA, 17 pages. https://doi.org/10.1145/3772318.3791666

1 Introduction

Software development platforms with large language model (LLM) integration now let users describe programs in ordinary prose and obtain runnable systems with minimal code editing. As of today, such services are no longer niche; the AI-enhanced platform Replit reports having "a global community of more than 40 million users" [72]; Lovable, which is a similar service, states that its users have created more than 10 million projects [31]; and Bloomberg reports that the LLM-based IDE Cursor surpassed one million users

*Theo B. Weidmann and Sverrir Thorgeirsson are co-primary authors.

This work is licensed under a Creative Commons Attribution 4.0 International License.
CHI '26, Barcelona, Spain
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2278-3/26/04
https://doi.org/10.1145/3772318.3791666

in 2025 [55]. The interaction style enabled by these and other platforms, dubbed *vibe coding* by former OpenAI employee Andrej Karpathy [37], allows one to rely on the behavior of the resulting program to iterate and improve the output program. As the capabilities of LLMs advance, this workflow is poised to become an even more common path for converting an idea into running code.

The shift toward language-first workflows raises questions about which skills matter most for future developers. For one, written communication skills may become increasingly important, in particular the ability to express intent unambiguously in writing. Unfortunately, this skill may currently be on the decline; for example, only 51% of high school graduates in the United States reach the ACT's English language benchmark for college readiness [5] (a low not seen in decades [2–4]), and adult literacy scores in OECD countries have been on a downward trend for several years [65]. To make sense of how this trend can affect current and future LLM-augmented software development, it would be helpful to know more precisely to what extent language skills, in particular written communication skills, relate to students' effectiveness at vibe coding, and how the predictive ability of this construct compares to traditional measures of programming skills, namely computer science achievement. The answer has direct implications for tool and curriculum design, for instance, on the design of systems that teach users to write better prompts, and whether software-engineering curricula should place greater emphasis on written communication.

To address this question, we conducted a preregistered[1], cross-sectional study with 100 tertiary-level students recruited from several universities in the same metropolitan area (see Figure 1). Each student completed a battery of tests to measure CS achievement, domain-general reasoning, written-communication proficiency, and proficiency in vibe coding. Our vibe coding tasks were designed iteratively by eight experts who offered their input through a structured consensus-building process, and we asked students to solve them on a custom programming platform that we built after analyzing attributes of commercial AI-based programming platforms. In developing the platform and the tasks, we chose to concentrate on GUI-oriented vibe coding: participants used natural language to describe small, web-based graphical applications and iteratively refined them, without ever viewing or editing the underlying source code. We chose this focus because GUI-based app creation is a primary use case for contemporary vibe-coding platforms, and because GUI-based programming is one domain of vibe coding that provides observable behavior that can be evaluated on its own.

The primary aim of the experiment was to examine to what extent written communication skills and computer science achievement predict performance in vibe coding, and how much these predictions are moderated by general-reasoning skills. We also sought to understand how the predictive ability of student attributes is impacted by the nature of the vibe coding task in question; specifically, whether decontextualized tasks (tasks that intentionally lack context and therefore cannot be succinctly described but require detailed, feature-by-feature prompts) demand a greater level of written communication skills. Additionally, in line with the *Standards for Educational and Psychological Testing* [8], which emphasize collecting validity evidence based on response processes, we analyzed students' prompts to test whether prompt quality serves as a mechanism linking writing skills to vibe coding performance.

The contribution of our work is threefold:

(1) **Empirical answer to which skills drive GUI-oriented vibe coding performance**. We provide a preregistered, cross-sectional study (N=100) that quantifies how written communication, computer-science achievement, and domain-general reasoning relate to performance in vibe coding, which we operationalized as programming through interacting with an LLM without concern for the underlying source code. We find that both CS achievement (r = .39) and writing skills (r = .29) significantly predict vibe coding performance, and that CS achievement remains a significant predictor after controlling for domain-general cognitive ability. In a joint model, CS achievement contributes roughly twice the unique variance of writing skills, though both add independent predictive value. The results have implications for both tool design and curriculum design (e.g., where to emphasize writing versus CS fundamentals).
(2) **A vetted assessment suite for GUI-oriented vibe-coding proficiency.** We curate and document an expert-designed set of natural-language programming tasks produced via a structured consensus process with eight subject-matter experts. The set is accompanied by clear scoring criteria used in our study, and is intended to support replication, comparison across cohorts, and possibly instructional use.
(3) **A research platform for LLM-native software creation.** We introduce a purpose-built environment that uses the core features of commercial AI programming tools while enabling controlled experimentation. The platform supports the iterative and behavior-guided workflow characteristics of vibe coding and is designed to support future empirical work on the construct.

The paper is structured as follows. In Section 2, we offer some additional context for our research and describe related work. We will then describe how the tasks we deployed in the study were designed (Section 3), how the survey instruments were designed and selected (Section 4), and how our vibe coding platform was designed and how it functions (Section 5). We will then describe the design and results of our pilot study (Section 6), which informed aspects of our primary study, whose methodology and results are described in Sections 7 and 8. We then describe our exploratory results in Section 9 (including an analysis of the user-submitted prompts), discuss our findings in Section 10, and conclude the paper in Section 11.

2 Background and Related Work

2.1 Vibe Coding

The idea of using natural language to write computer programs predates LLM-guided programming, and has long been the subject of debate and criticism [19, 28]. Several lines of work have pursued this ambition from different angles and with varying success. The early programming language COBOL had the ambition

[1]The preregistration can be found at https://aspredicted.org/wg3h-dx9m.pdf with additional materials mentioned in the preregistration located at https://doi.org/10.6084/m9.figshare.29509628.

to be "an English language programming system designed for non-professional programmers" [74]; Donald Knuth's "literate programming" paradigm [41] framed programs as "works of literature," with code organized for human readers and embedded in explanatory prose [41]; and various strands of research under the name of "natural-language programming" have explored translating constrained English into executable code, often via controlled subsets of English to avoid ambiguity [42]. However, only with the advent of LLMs has it become feasible to map conversational (and possibly underspecified) instructions to code.

LLM-augmented programming can take many rich forms, and not all implementations of the paradigm expect the user to communicate entirely in prose. For example, Liu et al.'s [46] workflow for spreadsheet programming converts a user's prose to code and then reflects it back as standardized, editable natural-language steps, letting programmers switch easily between code or prose, depending on the abstraction level they prefer. In the same vein, a recent paper on a system called *DirectGPT* [51] introduced the idea of allowing users to modify parts of the generated artifact both with natural-language prompting and direct manipulation on the visual output of the program, allowing the programmer to choose whether to focus on prose or the system output. However, many LLM-augmented development workflows support coding more as a chatbot interaction. Jiang et al.'s 2022 work [35], identified in a recent scoping review as the earliest academic paper on an LLM-based coding tool [43], found that users effectively treat natural-language prompts as a new programming notation, developing ad hoc "syntax" and debugging strategies for their prose instructions when working with a code-generating LLM. Although it is not clear whether this result still holds (other findings from that era have been conjectured to be "not as relevant anymore" because LLM output quality has significantly increased [11]), recent work continues to show that LLM-based programming tools can substantially reshape the way humans approach software development. For instance, Mozannar's 2024 study found that using LLM-based tools for programming, such as GitHub Copilot, can "significantly change user behavior" [57], and that developers differ in how they choose to use such tools: some write natural-language descriptions of individual functions, whereas others rely on more technical notation such as docstrings and function signatures. Taken together, these results suggest variation not only in the design of LLM-augmented programming tools, but also in how developers use them in practice.

More specifically, the interaction style in which the programmer does not interact with the code, instead relying on the LLM to generate a working program, was dubbed *vibe coding* [37] by OpenAI co-founder Andrej Karpathy in February 2025. Karpathy's formulation involves that one will "forget that the code even exists," and as such, vibe coding can be considered a variant of LLM-augmented programming that relies exclusively on iteratively writing prompts and evaluating program behavior. Consequently, it means that the act of writing turns into a primary programming skill: developers must translate design intent into precise, revisable prose instructions for the model. The viability of vibe coding means that building software is now possible for those without the technical skills needed to understand code, as several platforms that support vibe coding have made explicit; for example, Replit's blog post on vibe coding describes it as "a paradigm shift in app development" under which one no longer needs to "learn programming languages and syntax" or "understand complex technical concepts" [66], and Replit's CEO has claimed that 75% of its users never write a single line of code [50].

As a new term, the definition of vibe coding is still in flux and susceptible to semantic dilution; some define the term broadly as any form of AI-guided programming [15], while others insist that such definitions dilute the term and that vibe coding refers exclusively to "building software with an LLM without reviewing the code it writes" [83]. The Google Cloud documentation [24] posits that vibe coding comes in two variants. One is the "pure" vibe coding version, which is consistent with Karpathy's definition, while the other version means that the user understands the code generated by the AI and can refine it. In our study, we explicitly target this "pure" or "no-code" variant of vibe coding, as it provides a well-defined construct to measure: vibe-coding proficiency as the ability to iteratively specify, refine, and debug program behavior through text prompts and observed behavior alone. A more permissive workflow that exposes the source code would risk confounding this construct, as some participants might choose to edit the generated code directly through the LLM interface, effectively reverting to traditional programming.

Vibe coding has many potential use cases and application domains. For instance, using vibe coding to build an application with a GUI remains a primary use case for LLM-based commercial systems such as Replit and Lovable, which provide integrated support for the front-end web stack (HTML, CSS, and JavaScript). This setting can exploit the verification-generation asymmetry of vibe coding: GUI-based applications have observable behavior or design that can be easy for users to evaluate iteratively, but can be hard or time-consuming for a human to generate. Vibe coding is also used for specifying data transformations and data analysis, for example in a similar way that Liu [46] describes, with the use of designing formulas in natural language for spreadsheets, or even for applications where the primary object of interest is a physical artifact rather than source code; previous work has shown that LLM-augmented programming can be used successfully for programming robots [36]. In our work, since vibe coding is widely used for visual applications, as seen by the large number of platforms focused on this use case, we chose to scope our evaluation to GUI-oriented vibe coding. One of our tasks also involved algorithms, so our task suite captures a limited subset of more algorithmic-centric vibe coding; we return to this boundary condition in the discussion.

2.2 Skills Required for Vibe Coding

The role of prose writing for vibe coding makes it a salient research question to what extent writing skills play a role in the quality of the applications that are generated. According to the National Assessment Governing Board that serves the U.S. Department of Education, writing is a "complex, multifaceted, and purposeful act of communication" that can be accomplished in many environments and with many different tools [58]. Writing skills are measured by various criteria, for example in how it meets "audience, purpose and genre conventions" which can be assessed in relation to content criteria such as organization and structure, reasoning, evidence, mechanics, syntax and surface conventions used in the writing [9],

but also more simply in how successfully one is able to "explain in order to expand the reader's understanding" [58]. In our work, we are particularly interested in the latter, which includes the ability to write reasoned and logically structured content, as success in vibe coding is likely to be at least partially determined by the ability to explain intent to a non-human collaborator that does not share the writer's tacit context. For human-to-human software requirement engineering, the importance of this ability is well-understood; the IEEE International Standard mandates that requirements should be "stated simply" and be "easy to understand" [1], and the NASA System Engineering Handbook appendix on "How to Write a Good Requirement" offers specific language guidance, for example, to avoid indefinite pronouns that could lead to misinterpretation.

Studies on human-generated prompt quality to LLMs suggest that many non-experts lack robust strategies for the kind of technical writing required by this new workflow, exploring prompts "opportunistically, not systematically" and importing misleading assumptions from everyday conversation [92]. Feng et al.'s study of the CoPrompt system shows that even experienced programmers struggle with prompt clarity and level of detail when using natural language to program: participants often wrote vague, "unorganized" prompts whose relationship to the resulting code was hard to reconstruct, and needed tooling to make the prompts readable and reusable [21]. Furthermore, some studies have analyzed explicitly how the quality of text impacts performance on LLM-guided programming, though not, to the best of our knowledge, via direct, participant-level assessments of writing proficiency in a cross-sectional design. Lucchetti et al. [48] analyzed properties of novice-written prompts and found that information content, not technical vocabulary, best predicts success; substituting imprecise terms with more precise terms from existing prompts had little effect on how the LLM performed, but miswordings that affected the semantics of a prompt (e.g. "print" instead of "return") did affect pass rates. They also show that rewording without adding information rarely helps and is common in failure cycles. A recent work found that LLM performance on code tasks degrades significantly by introducing syntax and grammar errors, but also structural variations in human language, such as the use of interrogative instead of declarative statements [12].

The impact of computer programming or CS skills on LLM-assisted programming performance has been studied directly and indirectly in several other research papers. A 2025 work studied the performance of those with and without programming experience on communicating input-output transformation problems in natural language, both to an LLM and to another human [70]. Although the style of communication (e.g., a generous use of examples) was a more important attribute than expertise, the study also found that those with a programming background had a modest performance advantage. Similarly, a 2024 study [60] found that exposure to CS beyond an introductory course predicted higher success on a specific style of programming tasks. Participants were given input-output pairs that corresponded to CS1-style programming tasks and asked to write natural language prompts to an LLM to generate code. Similar to our study, participants could not work with the generated code directly, as the objective was to "[focus] on natural-language-to-code interactions." Unlike our study, the LLM system relied on a single-turn, stateless prompting, meaning that while participants could make unlimited attempts, they could not build on prior context. This design differs from typical LLM workflows, but may have introduced a tighter experimental control.

A correlational research study from 2023 [34] found that performance on a computational thinking test [82] (developed for middle school students but administered to adults) predicted performance on LLM-guided programming. The correlation was strong and significant, but had limited precision due to the low sample size ($N = 19$). As in our study, the participants were presented a visual task (a simple-grid based game) and were then asked to recreate it iteratively with the assistance of an LLM, and the quality of the program was measured with a functionality criteria. However, unlike our study, the authors note that the limitations of the online programming environment made it impossible to test a user interface; instead, the participants had to assess the functionality of the resulting program via text.

In general, language skills may have some connection to programming performance. A 2014 study [77] used functional magnetic resonance imaging (fMRI) to show that brain regions that are related to language processing are activated during programming, although this result was contradicted by later findings [33, 47]. English language skills have been found to correlate with software engineering achievement [6], and according to a 2020 study, adult learners' language-learning aptitude was a greater predictor of performance in Python programming than mathematics skills [71], which is consistent with older findings that verbal achievement is a greater predictor of CS grades than math achievement [44]. However, we are not aware of existing studies on the relationship between writing ability alone and CS achievement, LLM-augmented or otherwise.

It is likely that domain-general cognitive skills may influence both writing and CS achievement. According to a 2023 study on 282 undergraduate students, certain types of cognitive skills had a significant, positive correlation with performance on a CS achievement test (namely the SCS1, which we also used in our work) [25]. Furthermore, writing skills and cognitive skills have been found to be correlated; a study on 550 children found a large, positive relationship (Pearson's $r = 0.77$) between intelligence test results (WISC-IV FSIQ [81]) and writing test results (the WIAT-II's writing component [80]). While these results come from different populations, they suggest that general cognitive skills could be a shared factor behind all three outcomes. To address this for our study, we included a measure of cognitive skills as a control variable so that any links we observe between CS achievement, writing, and vibe coding skills are less likely to be explained simply by domain-general cognitive skills. In our work, we include cognitive skills only as a statistical control to reduce confounding, not as a statement about its instructional importance or a recommendation for educators to consider such scores in their instruction.

3 Task Design

As a measurable construct, vibe coding lacks a universal definition, task designs, and performance criteria. In our study, we chose to evaluate the construct by (a) granting the participants access to an LLM-based agent that writes code iteratively from human task specifications, and (b) asking the participants to solve two types

of tasks: replication and feature extension of existing programs that students could explore at will. This choice of task types was motivated by both practical and measurement considerations. For one, providing a textual specification would invite students to copy that text directly into the model or treat it as the definitive prompt, which would shift the construct to the near-trivial (or at least irrelevant) ability to copy or paraphrase text. Instead, our design requires the participants to formulate the target themselves, evaluating their ability to articulate a system design in such a way that an LLM agent can implement it. Broadly speaking, the ability to infer behavior from an existing system has many established use cases in software development and security [40], and has precedents in computing education, for example in reverse-engineering games [10].

To acquire the vibe coding tasks that the participants were meant to replicate or extend, we used a Delphi-style technique [45], namely expert elicitation using a two-stage structured process. Our goal with this process was to acquire two of the three tasks that we would use for our study. After securing ethics approval, we recruited eight participants for a task generation expert panel; all panelists were recruited based on their substantial expertise with LLMs and vibe coding. No compensation was offered. In addition to their familiarity with the domain, the experts were selected for their geographic and occupational diversity; the panelists were based in four countries and included three professors in the fields of CS education and human-computer interaction, one secondary school CS instructor, one postdoctoral researcher, one doctoral student, one master's student, and one front-end developer. Four of the panelists were women and four were men. To preserve independence, the authors of this paper did not serve on the panel.

In the first stage of the process, each panelist was asked to offer a full specification of a task that met the following criteria:

(1) **Interdisciplinarity**: The topic of the task should resonate beyond computer science, engaging at least one other domain (e.g., linguistics, civics, or art).
(2) **Real-world use case**: The task should be grounded in a plausible scenario that reflects a genuine need or common challenge encountered outside academic or very specialized contexts.
(3) **GUI-first implementation**: The resulting program should be used primarily through a graphical user interface (GUI); external APIs are acceptable, but no server-side programming should be required.
(4) **Non-trivial complexity**: The task should involve multiple components and decision points, so it is not trivial with LLM assistance, yet avoid advanced algorithms and keep mathematics at or below the secondary-school level.
(5) **Time-feasible**: Participants should be able to implement an end-to-end version within roughly 10–15 minutes using iterative prompting.
(6) **Clear specification**: A typical tertiary-level student should be able to grasp the app's purpose and required features quickly, minimizing time spent interpreting the brief.
(7) **Culture-fair**: The task should not assume particular cultural experiences or local knowledge, reducing construct-irrelevant variance and supporting fair comparison across participants.

We chose the first two criteria to keep the tasks authentic; in Schriebl et al.'s multidimensional model of authentic learning in science education [76], "real-world authenticity" is strengthened by content contextualization, especially societal relevance, connection to daily life, and content-related interdisciplinarity. The third criterion, that the tasks should possess a GUI, was important for several reasons; first, we find that a core aspect of vibe coding is the iterative shaping of how a program feels in use, which is inferred from its state and responsiveness. To achieve this, a visible interface is more helpful than to infer the state from code or console output. Second, guidance on authentic tasks also emphasize a visual product that students can share, a feature that is naturally supported by applications with a GUI, and third, we believe that vibe coding is often used for such applications in practice (for example on existing "vibe coding platforms"; see Section 5.1). The remaining criteria were designed to help us run a fair, timed study, keeping the focus on what participants do with LLM support, and reducing construct-irrelevant variance, such as task misreading.

In the second round of the process, each panelist was asked to rate each other's tasks anonymously on a five-point Likert scale using five scales from the earlier round. We chose to omit the third scale for this round as all provided tasks met this criterion, and the fourth and fifth scale were collapsed into one. After the second round was complete, we found three tasks that scored high on each scale (median rating ≥ 4 on all five scales) and whose ratings had a high consensus, i.e., the interquartile range (IQR) was ≤ 1 for at least 80% of the scales, making a third round unnecessary and likely to impose an unnecessary burden on the panelists. The average ratings can be seen in Table 1. Two of these tasks (Task 1 and Task 8) had a similar description, so we chose to combine them into one. Our panelists also offered comments on how each task could be improved, some of which we used to refine the tasks; for example, the meal planner application was missing an ability for students to select specific meals.

Lastly, to complement the two authentic tasks selected via the Delphi-like process, we designed a third decontextualized task that was intentionally non-authentic. The aim was to stress the construct of vibe coding as expressive specification: we wanted to determine if the participants would be more successful at implementing tasks with vibe coding without relying on task labels that carry strong priors for an LLM, such as a "meal planner" or a "scheduler." To that end, we designed an additional decontextualized task in the form of a small GUI toy application that was designed to have obvious behavior, but the labeling of the user interface was intentionally opaque as to not reveal a specific purpose.

Submissions were evaluated for behavioral fidelity using rubrics that we defined before the study took place. Participants were instructed that imitating the style and design of the base applications, such as the colors and dimensions of their components, was not necessary.

4 Survey Instruments

4.1 Written Communication Skills

Identifying a suitable instrument to measure adults' writing proficiency is challenging for two reasons: construct fit and deployment

Task	Interdisc.	Authenticity	Challenge	Purpose clarity	Culture fairness
Task 1: Exam scheduler	4.3 (1.0)	4.4 (1.0)	4.7 (0.5)	4.3 (1.0)	4.6 (1.0)
Task 2: Currency calculator	4.3 (0.5)	3.9 (2.0)	3.7 (2.0)	4.7 (0.5)	4.4 (1.0)
Task 3: Meal planner	4.6 (0.5)	4.1 (1.0)	4.1 (1.5)	4.6 (1.0)	4.6 (1.0)
Task 4: Meeting organizer	4.3 (0.5)	4.4 (1.0)	3.7 (1.5)	4.0 (1.5)	4.6 (1.0)
Task 5: Isometric room builder	4.1 (0.5)	3.3 (1.0)	3.7 (1.5)	3.9 (0.0)	4.7 (0.5)
Task 6: Eye tracker app	3.7 (2.5)	3.7 (2.0)	3.7 (2.5)	2.6 (2.0)	3.7 (2.0)
Task 7: Celebrity ranker	3.7 (1.0)	3.3 (1.0)	3.6 (1.0)	4.6 (1.0)	4.4 (1.0)
Task 8: Course platform	4.4 (1.0)	4.6 (1.0)	4.3 (1.0)	4.7 (0.5)	4.9 (0.0)

Table 1: The mean panelist ratings and interquartile ranges (in parentheses) of each proposed task according to our five criteria, namely interdisciplinarity, real-world use case (authenticity), non-trivial level of complexity (challenge), purpose clarity, and culture fairness. The tasks that were selected for the study (i.e., an exam scheduler, meal planner, and a course registration platform) are highlighted.

constraints. First, the literature documents a persistent gap in adult-focused, standardized measures [54]; many widely used batteries stop providing reference data beyond very early adulthood and/or use developmentally inappropriate prompts (such as WIAT-III's "what is your favorite game?" [30, 54]). Second, even for nearly acceptable instruments, practical considerations made them unsuitable for our study design; many instruments are proprietary and restricted to specific purchasers, or have licensing, privacy, and access issues, such as requiring participants to use an external website where we do not have control over user data.

For this reason, we enlisted two instructors in written communication from the *Language Center of UZH and ETH Zurich* to design a task-based assessment and an analytic rubric for adult writers based on assessments used in their course. The instructors were volunteers, not a part of the author team of this paper, and were not compensated for their help. The rubric they designed can be found in Figure 2. The essay prompt is:

> *Write a 300-to-450-word description, divided into 2 to 3 paragraphs, about one technical concept of your choice that you are familiar with from your current university studies. Your description should be written for a college-educated but non-expert adult.*

To understand the construct coverage of this assessment, we consulted with staff responsible for AAC&U's VALUE initiative [53, 63] about the Written Communication VALUE rubric [9], which has been used by many universities in the United States. While the VALUE rubric was not a good fit for our needs as it is designed for research-supported writing and institution-level assessment rather than grading, the criteria of the rubric (context and purpose, content development, genre/disciplinary conventions, sources and evidence, and control of syntax and mechanics) is close to the one we used. This conceptual overlap, together with VALUE's explicit expectation that campuses translate and adapt rubric language to local contexts, lends some support for the content validity of our locally developed instrument.

To grade participant-written essays according to the rubric, two independent graders rated each essay blindly and independently; one professional editor and technical writer, and one graduate assistant. We held two calibration meetings for the graders to clarify questions on the rubric, such as how to grade single-paragraph essays. Per our preregistration, we targeted inter-rater reliability of at least $ICC(2,2) \geq 0.75$. When the two ratings differed by more than ten points, a third blinded rater (a professor in linguistics, also independent) scored the essay, and the final score was the mean of the two closest ratings (otherwise, the mean of the first two was used). Our pilot study helped us settle the final wording of the prompt and the amount of time allotted, which we then held constant for all participants.

We note that we use the rubric to index a domain-general proficiency in written communication. Individual criteria, such as paragraph organization and cross-sentence cohesion, are treated as reflective indicators of that proficiency. We do not claim that any specific feature (e.g., use of anaphoric nouns) is itself beneficial for LLM interaction; rather, we test whether the underlying ability to structure and convey technical information in prose predicts performance on LLM-mediated programming tasks.

4.2 Computer Science Achievement

To evaluate CS achievement, we conducted a search of validated instruments used in computing education. We sought an established instrument that made minimal assumptions about participants' prior programming experience, i.e., which specific programming languages they had been instructed in. Ultimately, we chose to measure the construct using a 12-item subset of the SCS1, a validated, language-independent (pseudocode-based) assessment of CS knowledge developed by Parker et al. [67]. Since its inception, SCS1 has become an important benchmark for measuring CS learning; a 2021 review identified 44 studies that have utilized this assessment in varied research contexts [68].

The 12-item subset that we used was the same one that Parker et al. [69] used in another correlational, cross-sectional study to measure CS achievement. In choosing this subset, we were also motivated by Parker et al.'s rationale, namely to maintain the content coverage of the full 27-item test by including four items each on definitional knowledge, code tracing, and code completion, and to

(1) **Overall effectiveness for the target readership**
   (a) The scope of the topic is appropriate for the length and timing of the exercise.
   (b) The topic is explained with an appropriate level of complexity (i.e., not over-simplified, but not too detailed).
   (c) New or difficult terminology is adequately explained.
   (d) The average target reader would understand this text.

(2) **Paragraphs**
   (a) The paragraphs address topics in a logical order.
   (b) Each paragraph has either a topic sentence or another clear organizational structure.
   (c) Each paragraph addresses a unified, cohesive topic.
   (d) There are visible and logical transitions between paragraphs.

(3) **Sentences**
   (a) Sentences are clear and straightforward, with subjects and verbs that can be easily identified.
   (b) The majority of sentences use active verbs that appear in the first half of the sentence.
   (c) The writer varies sentence structures, including both complex and simple sentences.
   (d) The writer varies sentence length.

(4) **Flow between sentences**
   (a) Topics follow a logical order from sentence to sentence within each paragraph.
   (b) The links between sentences are clearly identifiable (i.e., no unclear pronouns, unclear continuations, etc.).
   (c) The beginning of most sentences has a clear link to the previous material.
   (d) A range of cohesive devices (e.g., repetition, substitution, anaphoric nouns, etc.) is used flexibly and accurately.

(5) **Language-level concerns**
   (a) The vocabulary is appropriate (in terms of complexity) for the target audience.
   (b) The vocabulary and grammar are accurate and precise enough for the reader to easily understand the text. Errors are minor and rare and have minimal impact on clarity.
   (c) The vocabulary and grammar are varied.
   (d) A wide range of lexical and grammatical structures is used accurately and flexibly to convey clear and precise meanings.

Figure 2: The grading rubric of written communication skills that we deployed in the study after expert consultation. Each sub-item is graded on a scale from 1 to 5, resulting in a maximum score of 20 for each category, and a maximum score of 100 in total.

preserve psychometric quality by selecting items according to their previously estimated item-response-theory difficulty and discrimination values. Furthermore, we believed that administering the full SCS1 in addition to the other tasks and surveys would induce too much testing fatigue, as the SCS1 is meant to take an hour to complete [68]. Hence, for the subset, we chose to impose a 25-minute time limit, which is proportional to its length. To ensure that, in doing so, we would avoid construct-irrelevant speed, we gathered validity evidence of the timed administration of the assessment in our pilot study.

4.3 General Reasoning Skills

To measure domain-general cognitive skills, we used an assessment instrument called the International Cognitive Ability Resource (ICAR) that was developed as an open alternative to proprietary tests [16]. The test consists of multiple-choice questions that involve verbal reasoning, matrix reasoning, letter series, and three-dimensional rotation. Compared to licensed options that also measure cognitive skills, ICAR is attractive to researchers as it requires no licensing fees, and its public availability (upon request) increases research transparency and reproducibility.

ICAR consists of 60 questions, but its developers also constructed a 16-question subset of the measure called the ICAR16, which contains four questions of each type [16]. Empirical studies support the ICAR16 as a valid indicator of domain-general reasoning; the test shows strong convergence with comprehensive cognitive batteries, with one cross-sectional study finding a large correlation ($r = 0.81$) between scores on ICAR16 and the proprietary Wechsler Adult Intelligence Scale (WAIS-IV) [89]. The instrument has also been found to be age and sex-invariant [90]. ICAR16 has been deployed in a number of research contexts; for instance, it has been deployed in genetics [84] and mental health [39] research.

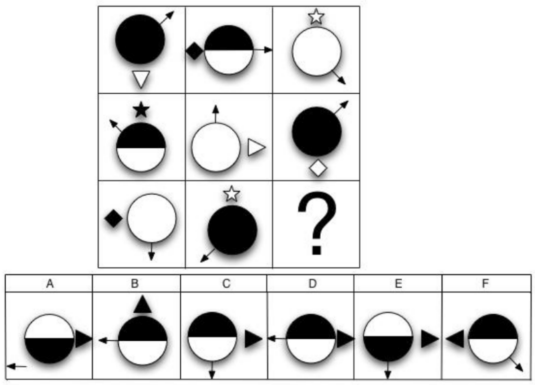
The ICAR16 should be administered with no more than a 16-minute time limit in studies involving young adults, and some researchers use as little as 10 minutes [20]. To find a suitable timer, the ICAR designers recommend piloting the test with a "handful of participants in your target population" [32], which is advice that we followed for our study (see Section 6). After our pilot, we chose a time limit of 12 minutes in our main study. The order of the 16 items was randomized as in Condon and Revelle's initial deployment [16].

5 Vibe Coding Platform

We developed a dedicated vibe-coding platform to guarantee reproducible access, satisfy privacy requirements, and to give us greater control over how to instantiate vibe coding. Reproducibility concerns included version drift, evolving interfaces, commercial paywalls, and the risk that services change or disappear during a study. We also wished to meet our ethics committee's requirements for robust data protection; third-party platforms provide more limited transparency and rely on varying data-handling policies. Importantly, we had to ensure that the provider of the AI model integrated into our tool does not retain participant inputs and that the model is not trained on participants' prompts.

Question 12 from ICAR16

Please indicate which is the best way to complete the figure below.



Answer options: (1) A (2) B (3) C (4) D (5) E (6) F

Question 9 from SCS1

Given the following code segment.

```
array = [3, 6, 8, 1, 2, 0, 7, 2, 9]
i = 0
odd = 0

WHILE (i < length(array)) AND (array[i] != 0) DO
    IF (array[i] % 2) == 1 THEN
        odd = odd + 1
    ENDIF
    i = i + 1
ENDWHILE
```

What are the values of the variables `i` and `odd` after the while loop completes its execution?

A. i = 1; odd = 0
B. i = 5; odd = 2
C. i = 5; odd = 3
D. i = 8; odd = 4
E. i = 8; odd = 5

Figure 3: Items from assessment instruments ICAR16 (left) and SCS1 (right) that were provided as examples in the ICAR guidelines [32] and by Parker et al. [67], respectively.

5.1 Survey of Prominent Vibe-Coding Tools

To ensure that our research platform was informed by the interaction flow and visual layout of existing tools, we first analyzed four commonly cited platforms associated with vibe coding. In addition to direct use of LLM chatbots such as ChatGPT, prominent tools include Replit, Cursor, Lovable, and Bolt [23, 75], which have also been highlighted in articles by major news outlets such as the New York Times and the Wall Street Journal [73, 78]. To compare them systematically, we asked each platform's AI agent to implement the same toy application of a simple password manager. This task was chosen because it requires both interface design and code generation, yet remains small enough to complete within a single session. While using each tool, we performed screen recordings and noted how the system supported iteration, debugging, and user control.

Across the four platforms we examined, the user interfaces followed a highly similar structure. Each relied on a chat-based interaction with the AI agent, coupled with a live view of the application under development. In Replit, Lovable, and Bolt, the chat window was placed on the left, with the application preview occupying the remainder of the screen. Cursor differed slightly: marketed as an "AI Code Editor" [17] and built on top of Visual Studio Code [18], it displayed the code editor on the left and the chat interface on the right. The four systems made it possible to view the source code if the user so desired, and, depending on the subscription plan, some of them allowed one to also edit the underlying code. We intentionally diverged from commercial tools here, as exposing the source code would have introduced an additional interaction mode and risked confounding our construct of "pure" vibe coding.

5.2 Study Platform Design

In contrast to the prominent platforms we evaluated, our own platform deliberately conceals the code entirely, consistent with the original definition of vibe coding [37]. As shown in Figure 4, the interface retains the chat window on the left and the app preview on the right, following the conventions of existing tools. To support our study design, we extended the left-hand sidebar to also display the task description and a timer, allowing participants to track the remaining time without distraction, and a button to show the sample application stimulus.

We also integrated a set of convenience features that were common across the tools we analyzed. These included the ability to roll back the application to previous iterations, and a live stream of the model's output to provide immediate feedback that the agent was actively working. In our implementation, the live stream was deliberately blurred, preventing participants from actually reading or copying code while still conveying progress (see the bottom left in Figure 4).

Once the tasks were finalized (see Section 3), we evaluated system performance across different LLMs from different vendors. In our tests through our custom-built platform, Claude Sonnet 4 consistently outperformed other models on the study tasks, delivering faster responses while maintaining high output quality. On this basis, we selected Anthropic Claude Sonnet 4 (20250514 release) as the model underpinning our platform. For implementation, we opted for a single self-contained HTML file including JavaScript and CSS, simplifying deployment and minimizing overhead. To further improve throughput, we introduced a diff-based update mechanism: instead of rewriting the entire file, the model was instructed to return only the code fragments that required modification in response to user prompts.

We ensured that the sample applications presented to participants during the study can be replicated fully by participants by also using our own tool to build the sample applications according to the task specifications (see Section 3). While reviewing the sample applications, participants further benefited from a feature-tracking mechanism: We instrumented the sample applications with additional code manually after construction through the vibe coding

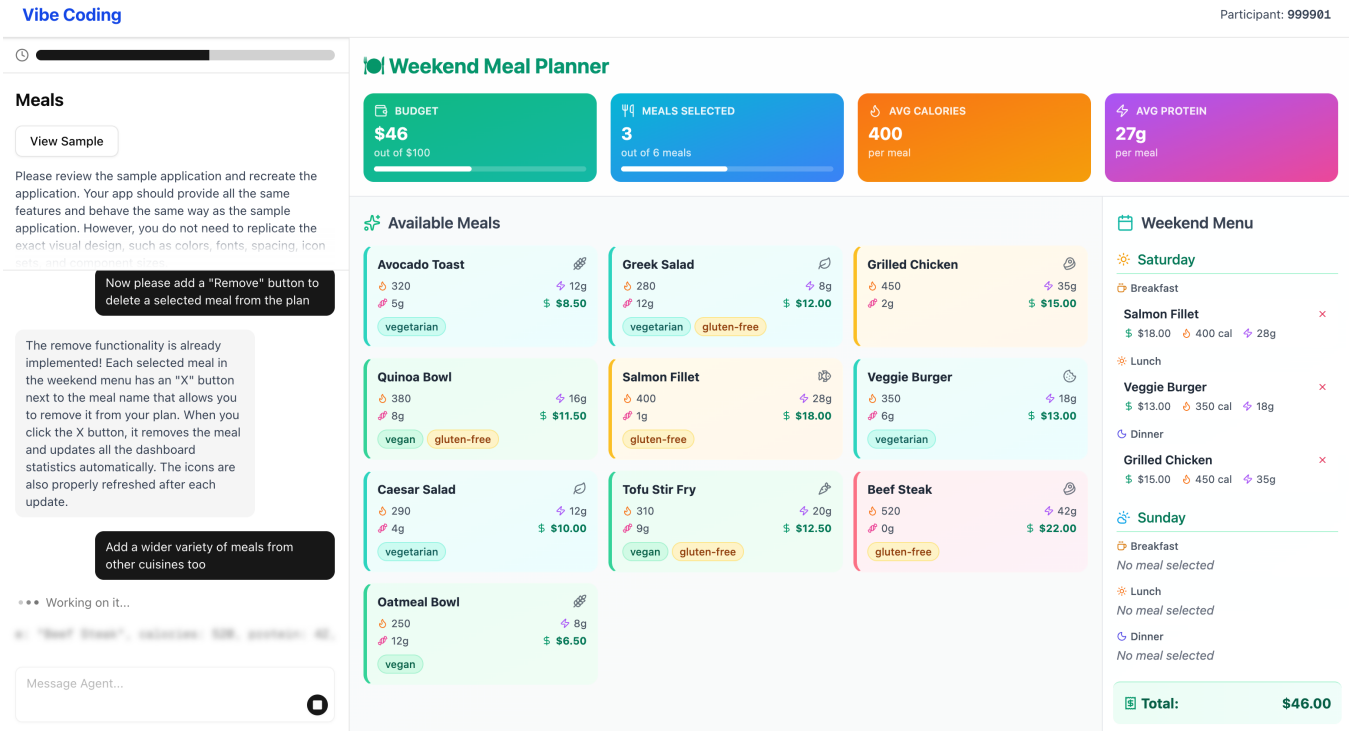


Figure 4: User interface of the tool we used in the study. The interface follows the vibe coding paradigm by not exposing any code. It features a chat box on the left, consistent with other tools, and an app preview on the right. The left sidebar also includes the task description and a timer showing the remaining time for participants (hovering over the bar shows the exact time remaining). The interaction seen in the screenshot was recreated based on prompts from participants during the study.

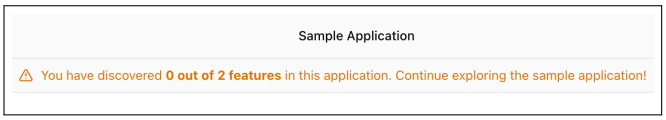


Figure 5: While exploring the sample application, participants are provided with feedback about the number of relevant features they have interacted with.

platform to track if and when a feature is used. We then displayed feedback to the user to indicate if they tried all features already, as can be seen in Figure 5. This information was also logged for further analysis.

6 Pilot Study

We conducted a pilot evaluation with 10 students who were recruited from the same pool as the main body of participants. The primary objective of the pilot evaluation was to determine a sound time limit for each of the three assessment instruments and to test our vibe coding platform. The pilot study had the same structure as our main experiment; all participants filled out the demographic survey first, after which they filled out the questionnaires in random order and then solved the vibe coding tasks on our platform. To limit learning effects, half of the participants, selected at random, submitted the vibe coding tasks first before they were asked to respond to the questionnaires. However, unlike the main study, the pilot essay prompt had a word limit of 150 words.

For the SCS1 12-item subset, the results indicated a high internal consistency: Cronbach's alpha = 0.83, Feldt 95% CI = [0.62, 0.95], meaning that even the lower confidence bound (0.62) is above the minimum acceptable threshold (0.6) for exploratory scales [26]. 9 of 10 participants completed all 12 items, suggesting only minimal evidence of speededness, and 96% of all questions were answered. For ICAR16, we gave the participants a 16-minute timer in the pilot test. 8 of 10 participants reached their last item (placing the test right at the 80% threshold for speededness [14]) and 90% of all tasks (144/160) were answered. Only 3 participants needed the full timer; 7 terminated the survey before the timer expired. For the essay task, 9 participants wrote full essays; one participant terminated the task after only a few minutes and after writing only 39 words.

For the SCS1 12-item subset, we retained the pilot time limit for the main study. Because this is the first administration of this instrument with an explicit timer, and because the pilot showed strong internal consistency and minimal speededness (9/10 finished; 96% of items answered), we believed that altering the limit would introduce unnecessary variance. For ICAR16, we reduced the limit from 16 to 12 minutes, as this remains within common ICAR practice (10-16 minutes) and most participants finished before the 16-minute

cap (7/10). Finally, for the essay task, we raised the target length to 300-450 words (as 150 words was much too short for substantive responses) and extended the time to 20 minutes. Additionally, for all surveys, we decided that participants would not have the option of exiting each assessment early.

While analyzing student solutions on the vibe coding platform, we observed that some participants were unaware of base-application features relevant to the tasks they were asked to implement or extend. To preserve construct validity, i.e., the ability to vibe code and not the ability to understand how an existing application works, we updated the platform so that users were explicitly notified of untested features in the sample application (in a small banner at the top of the sample application window, see Figure 5).

7 Method

7.1 Objectives

We preregistered three research questions about the role of written communication skills for vibe coding:

(1) **RQ1: Task-type specificity.** Does written-communication proficiency relate more strongly to performance on the decontextualized task than to performance on the two context-rich tasks (replication and feature addition)?
H1. The association between written-communication proficiency and performance will be larger for the decontextualized task than for each context-rich task.
(2) **RQ2: Overall association and independence from general reasoning.** (a) Are written-communication skills positively correlated with vibe coding performance? (b) Does this correlation remain after statistically controlling for domain-general cognitive skills?
H2a. Written-communication skills will correlate positively with average performance across the three tasks.
H2b. No directional hypothesis is specified regarding the partial association after controlling for domain-general reasoning.
(3) **RQ3: Relative predictive value.** In predicting vibe-coding task performance, how do written-communication skills compare with CS achievement?
H3. No directional hypothesis is specified about which predictor will be larger.

7.2 Participants

After receiving ethics approval from our institution, ETH Zurich, we conducted a cross-sectional laboratory study with the help of the Decision Science Laboratory (DeSciL) at ETH. DeSciL maintains a large volunteer pool of students from ETH and the University of Zurich, from which we recruited our participants. Each participant received a flat cash payment of 55 CHF for a single session that lasted 1 hour and 45 minutes in the DeSciL controlled computer laboratory. To participate in the study, the students had to have completed an introductory course in computer science, have prior experience working with LLMs for solving computer programming tasks, and meet at least a C1-level of proficiency in English according to the Common European Framework of Reference for Languages [64]. We chose to recruit 100 participants to achieve sufficient power to at least calculate an overall association between the constructs; a priori power analysis for Pearson's r indicated that a sample of N=85 would be sufficient to detect a medium-sized association (r=0.30) with 80% power in a two-tailed test at α=0.05.

After signing consent forms, the participants completed a brief demographic survey on their age, gender, and field of study. We also asked participants how often they used large language models and the extent to which they perceived them as helpful in daily life. Participants then completed vibe coding tasks and a battery of survey instruments in counterbalanced order (see Figure 6).

7.3 Analysis Plan

The primary measures in our study are (i) written-communication skills, (ii) CS achievement (SCS1 12-item subset score), (iii) domain-general cognitive skills (ICAR16 scores) and (iv) vibe coding performance. All of these measures are normalized to a $[0, 1]$ range. We answered our research questions in the following way:

(1) **RQ1**: We implement a mixed-effects model to test whether written-communication skills relate differently to performance across vibe task types, with the Writing x Task-Type interaction as the primary confirmatory test (α = .05). In contrast to our preregistration, we first compare the correlation between the task types and written communication before deciding whether to construct the model.
(2) **RQ2a**: We compute the Pearson correlation between written-communication proficiency and vibe coding performance, reporting r, a 95% confidence interval (Fisher z), and a two-tailed p value.
(3) **RQ2b**: We fit an ordinary-least-squares (OLS) regression of aggregate performance on written-communication proficiency and ICAR16 (both z-scored). We report the writing coefficient with 95% CI, standardized beta, and semi-partial R^2 for writing.
(4) **RQ3**: We conduct hierarchical OLS regressions to compare unique variance: (a) We enter the CS achievement score, then add writing and record $\Delta R^2_{\text{write}|\text{CS}}$, (b) Reverse the order to obtain $\Delta R^2_{\text{CS}|\text{write}}$. We compare the two ΔR^2 values to describe which predictor adds more unique variance; standardized final-step coefficients with 95% CIs are also reported.

For the grading of the written communication assessment, we refer to the previous sections. For the grading of the vibe coding tasks, one member of the author team manually and blindly reviewed each application and scored each core feature (as defined on the predefined grading rubric) on a 4-point Likert scale, with anchors defined as *not acceptable*, *below expectations*, *meets expectations*, and *exceeds expectations*, as proposed in the literature [56].

8 Results

8.1 Overview

Of the 100 students who participated in the study, the demographic survey indicated that there were 56 women, 44 men, and no agender or nonbinary participants. The average age of 99 of those participants was 25.0 (SD = 3.5); one participant did not enter a number when asked about their age. 51 participants were engineering and technology students, 20 were from the natural sciences, 11 were from the social sciences and the humanities, and the remaining

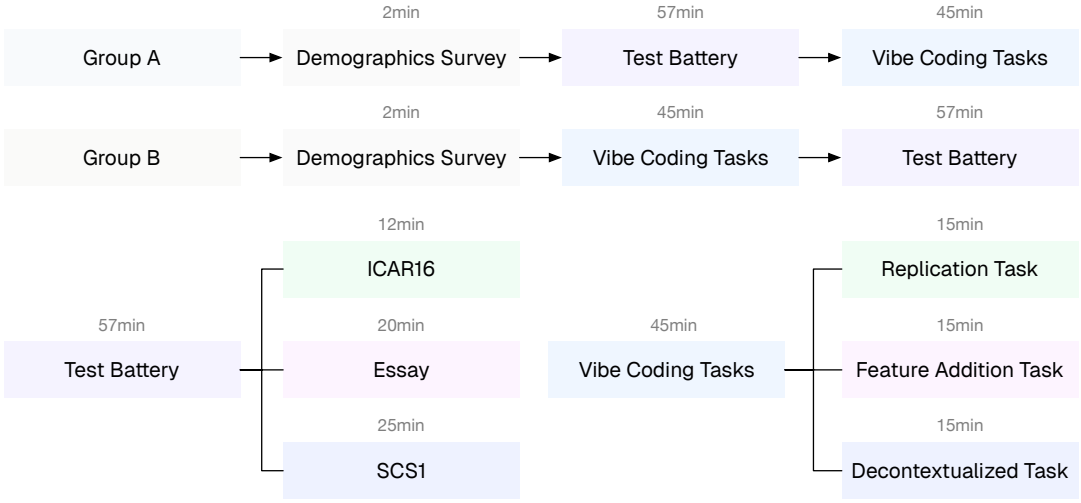


Figure 6: A diagram depicting the flow of our experiment. Participants were randomly divided into Group A and Group B. The order of the instruments in the test battery and the vibe coding tasks was randomized for each participant.

Instrument	Mean (SD)	Min–Max
SCS1	0.53 (0.24)	0.08–1.00
ICAR16	0.56 (0.18)	0.19–1.00
Writing Skills	0.72 (0.11)	0.30–0.97
Vibe Coding Tasks	0.45 (0.19)	0.00–0.83
Replication Task	0.46 (0.32)	0.00–1.00
Feature Addition Task	0.55 (0.23)	0.00–1.00
Decontextualized Task	0.34 (0.26)	0.00–0.83

Table 2: Descriptive statistics for the results from our study instruments, showing mean, standard deviation (SD), and the minimum and maximum scores achieved. All scores were normalized to [0, 1], with 1 denoting the maximum score possible.

	CS	Cog.	Writing
Cog.	0.417* (p<.001)		
Writing	0.126* (p=.213)	0.365 (p<.001)	
Vibe	0.386* (p<.001)	0.352* (p<.001)	0.290* (p=.003)

(a) Pearson correlations

	CS	Writing
Writing	−0.032 (p=.757)	
Vibe	0.281* (p=.005)	0.186 (p=.066)

(b) Partial correlations controlling for cognitive ability

Table 3: Correlation matrices between CS achievement (CS), cognitive ability (Cog.), writing skills (Writing), and vibe coding performance (Vibe). All values are Pearson's r with two-tailed p-values in parentheses, with an asterisk (*) indicating statistical significance ($p < 0.05$). Partial correlations (b) control for cognitive ability.

students were from other fields. Our system logs show that for all but 3 of the 300 vibe coding task submissions, participants had explored all features in the sample application before submission.

After grading the 100 essays, our essay graders reached an ICC(2,2) consensus value of 0.735. In line with our preregistration plan, this prompted a round of recalibration and rescoring, as the value was below the 0.750 threshold. We asked the two graders independently to take another look at their own grades and verify if any should be regraded to maintain consistency with their other grades; this lowered the ICC(2,2) value to 0.731. At this point, we terminated the process, as future rounds seemed unlikely to make a difference, and because the value we arrived at is above the widely cited acceptability cutoff of 0.7 for reliability coefficients [61] (and above the "good" cutoff of 0.6 for ICC in particular [27]). After this, 21% of the essays differed by >10% of the grading scale, and for those essays, the difference was resolved by our third grader as we specified in our plan. The descriptive statistics for the essay grades and the other primary constructs are shown in Table 2.

Cronbach's alpha for the SCS1 grades was 0.734, which is above Nunnally et al.'s acceptability threshold, but Cronbach's alpha for ICAR16 was 0.625, which is below the 0.7 threshold but above the threshold of 0.6 for exploratory research [26]. Because of this, we used the ICAR16 measure primarily as a covariate and did not analyze ICAR16 item-type subscales such as verbal reasoning.

We computed the zero-order correlations between the four primary constructs we measured (Table 3-a). All correlations were positive and statistically significant ($p < 0.05$), except for the correlation between CS achievement and writing skills, which was small and non-significant. In particular, both writing skills and CS achievement correlate with performance on vibe coding. After controlling for performance on the domain-general cognitive test (Table 3-b),

Task	r	p
Context-Rich Task 1 (Replication)	0.154	0.126
Context-Rich Task 2 (Feature Addition)	0.245*	0.014
Decontextualized Task	0.239*	0.017

Table 4: Pearson correlations between the three vibe coding task scores and writing performance (two-tailed p). As specified in Table 3, the overall Pearson's r between the task average and writing was 0.290.

the correlation between CS achievement and vibe coding skills remained significant (Pearson's $r = 0.281$, $p = 0.005 < 0.05$), but not the correlation between written communication and vibe coding skills (Pearson's $r = 0.186$, $p = 0.066 > 0.05$).

8.2 Research Questions

For **RQ1**, we asked if written-communication skills relate more strongly to performance on the decontextualized task than to performance on the two context-rich tasks. A simple check showed that we could not reject the null hypothesis, as the correlation (Pearson's r) between writing skills and the decontextualized task (0.239) was lower than for the feature addition task (0.245; see Table 4).

For **RQ2**, we asked whether written-communication skills are positively correlated with vibe coding performance (a relationship that we hypothesized exists) and whether this correlation persists after controlling for domain-general cognitive skills, for which we made no hypothesis. In accordance with our preregistration plan, we computed Pearson's r for the former and found a correlation of $r = 0.290, p = 0.003$, 95% confidence interval of $[0.099, 0.460]$, indicating a significant but small (< 0.3) positive correlation, so we could reject our null hypothesis. This correlation becomes non-significant when controlling for the ICAR16 results: partial $r = 0.186$, 95% confidence interval of $[-0.012, 0.386]$, $p = 0.066$.

For **RQ3**, we asked how the predictive value of CS achievement and written-communication skills compare on vibe-coding performance. We compared the unique variance in aggregate vibe-coding accuracy explained by written-communication proficiency and CS achievement using hierarchical OLS, per the preregistered plan. Entering the CS achievement score first and then adding writing increased R^2 from 0.150 to 0.208 ($\Delta R^2_{\text{write}|\text{CS}} = 0.059$). Reversing the order, entering writing first and then adding SCS1 increased R^2 from 0.083 to 0.208 ($\Delta R^2_{\text{CS}|\text{write}} = 0.125$). Thus, CS achievement scores contributed roughly twice the unique variance of writing scores in predicting aggregate performance, although writing still added nontrivial incremental variance. Using both predictors in the final model, standardized coefficients indicated positive associations for both: Writing had the parameter $\beta = 0.244$ [95% CI 0.063, 0.425], $p = 0.009$; and CS had the parameter $\beta = 0.356$ [95% CI 0.176, 0.537], $p < 0.001$. Descriptively, CS achievement is the stronger predictor of aggregate vibe-coding accuracy, but written-communication skills also show a reliable unique contribution beyond CS achievement.

9 Exploratory Analysis

As the outcomes on the CS achievement test and vibe coding tasks were significantly and moderately positively correlated, and the correlation remained significant even when controlling for domain-general cognitive skills, we analyzed the secondary data to explore what could contribute to this relationship. One exploratory hypothesis that we developed was that the effect was moderated by experience and interest; namely, that students with more CS experience and hence greater CS achievement might be more familiar with how to work with large language models (LLMs). To investigate this, we computed how the participants' self-reported Likert-scale frequency of LLM usage (encoded as a 1-5 variable, with 5 mapped to the highest frequency) correlated with the vibe coding performance. To our surprise, we found a significant negative correlation ($r = -0.258; p = 0.010$). We also found a significant negative correlation between frequency of LLM usage and written communication skills ($r = -0.282, p = 0.005$), but no correlation between LLM usage and CS achievement ($r = 0.001, p = 0.994$) or LLM usage and the ICAR16 results ($r = -0.070, p = 0.486$).

A second exploratory question that we considered was whether the writing assessment primed participants to write better prompts for the vibe coding tasks. To analyze that, we computed a two-sided independent-samples Student's t-test on the vibe coding performance of students who began the study with the survey battery versus those who began the study with the vibe coding tasks. We found no evidence of a difference; the means were 0.446 and 0.448, respectively, with $p = 0.953$.

Our third exploratory question, designed to better understand the robustness of our results, was on how the quality of the user prompts correlated with the essay grades and performance on our vibe coding tasks. To measure the prompt quality, we followed a two-step approach:

(1) First, we organized a human evaluation of the writing quality of the prompts. We asked the creators of our essay grading rubric to design a grading rubric for the writing quality of prompts for human-LLM interaction. They designed a rubric (included in our supplementary materials) that involved coherence, task-appropriate complexity, and instructional clarity, bearing in mind that prompts tend to be expressed in the imperative grammatical mood, and that many elements of essay writing (such as sentence flow) are less relevant in this context. We then recruited a member of the vibe coding expert panel to grade the prompts using this rubric, who graded the full prompt sequence for each task attempt as a single unit. The rubric was designed by a team that had not seen our prompt data. The expert panelist, who was not involved in any previous rounds of grading, graded the prompts blind to all other participant data, including essay scores, task outcomes, and the generated code.
(2) Second, to triangulate the results from the human evaluation, we considered the *lexical richness* of the prompts, a term "also known as lexical diversity" [88], which has been found to be significantly correlated with writing quality [85, 87, 88]. Many automatic measurements of lexical richness exist; some measures involve counting the frequency

	Essay grade	Essay LD	Vibe perf.
HD-D	0.206^{*} (p=.0399)	0.208^{*} (p=.0379)	0.310^{*} (p=.0017)
MTLD	0.248^{*} (p=.0129)	0.258^{*} (p=.0095)	0.343^{*} (p=.0005)

Table 5: Pearson's r between prompt-related measures and outcome variables. HD-D and MTLD measure the lexical diversity (LD) of students' prompts. Essay LD refers to the lexical diversity of the student's essay. All correlations are marked with an asterisk as they were significant ($p < 0.05$).

of uncommon words in the text (which is called lexical sophistication), while others quantify the balance between vocabulary variety and text length. However, lexical sophistication may not be suitable or meaningful in our context; first, it would require an appropriate reference corpus, and second, prior work has found that when substituting individual words in human-generated prompts with synonyms, the rarity of the synonym does not correlate with improved output quality from the LLM [79]. Instead, following McCarthy and Jarvis [52], we chose two common measures that are length-independent and applicable to short texts; the HD-D, a probability-based measure that captures how much vocabulary variety appears across random word samples drawn from the text, and the MTLD (Measure of Textual Lexical Diversity), which measures how long, on average, the text sustains varied vocabulary before words start repeating. Both measures have been validated as robust alternatives to simpler metrics like type-token ratio, which is known to be confounded by text length [52]. We used the `lexical-diversity` library in Python to calculate this.

We then computed how a student's average prompt quality measure correlated with other metrics. All correlation coefficients were positive and significant; students who exhibited varied vocabulary in their prompts were also significantly more likely to (i) exhibit varied vocabulary in their essays, (ii) score a higher grade on their essay, and (iii) achieve a greater performance on the vibe coding tasks (see Table 5). Students who exhibited higher human-graded prompt quality were also significantly more likely to have a higher grade on their essays and to score better on the vibe coding tasks (Pearson's $r = .353$ and $r = .479$, respectively, with $p < .001$).

To examine whether the observed associations are consistent with prompt quality serving as a linking mechanism between writing skill and vibe coding performance, we used the human-graded prompt quality to conduct mediation analysis, which is a statistical technique that has sometimes been applied in CHI research papers [13, 91]. Following Hayes' textbook on the subject [29], we used 10,000 bootstrap resamples. As we noted before, writing skill was associated with prompt quality (path $a = 0.35$, $p < .001$), and prompt quality was associated with vibe coding performance when controlling for writing skill (path $b = 0.43$, $p < .001$). The total association between writing and vibe coding (path $c = 0.29$, $p = .003$) was reduced to non-significance when prompt quality was included in the model (path $c' = 0.14$, $p = .145$). The indirect effect was 0.152, with a 95% bootstrap confidence interval of $[0.061, 0.279]$ that excluded zero, and accounted for approximately 52% of the total association. These results are consistent with prompt quality serving as an intermediate variable linking writing proficiency to task performance: participants with stronger writing skills produced clearer, better-organized prompts, which were in turn associated with more successful outcomes. As we conducted a cross-sectional study, we do not wish to draw causal conclusions; rather, this pattern provides response-process evidence supporting the construct validity of our vibe coding measure. Rather than capturing some ability unrelated to prompting, the behavioral outcomes appear to reflect differences in how effectively participants specified their intent to the LLM.

10 Discussion

We believe that the primary and most interesting finding from our study is that both written communication skills and CS achievement contribute positively, significantly, and independently to GUI-oriented vibe coding performance, and in the case of CS achievement, in a way that cannot be explained by domain-general cognitive ability alone. Furthermore, our exploratory analysis suggests that this relationship is not explained either by high-performing CS students having had greater experience in working with LLMs, and because the study was designed in such a way that the code generated from the LLM was not visible (only the rendered output itself), we know that CS students had no obvious advantage in interpreting the model output. As our study is correlational, we cannot make causal claims about the impact of CS achievement on vibe coding; such claims could only be investigated in controlled studies. On the other hand, our study results can be used to generate hypotheses for causal mechanisms that could be explored in such studies. For instance, it is possible that the structured thinking that students learn in computer science classrooms lends itself well to specifying program behavior; vibe coding may draw, in some sense, on core CS knowledge or parts of the "hidden curriculum" of CS, such as problem decomposition and algorithmic thinking, which we believe is a part of CS achievement. It is also possible that a greater command of CS-related vocabulary can help students write better prompts.

As for written communication skills, we believe that students with high scores on the construct are not only skilled at clear writing, but also at following conventions for structured writing. Such students may therefore also be skilled at structuring their prompts in a way that helps them accomplish specific tasks. In the current educational environment, where U.S. 4th grade reading scores are still well below pre-pandemic levels [22, 59], and both PISA scores in reading performance and adult literacy scores are trending downwards [62, 65], it is plausible that any positive impact of language training on students' ability to convert their ideas into code could motivate students to learn to express their ideas more clearly.

The implications of our study could also inform tool design. A significant number of tools are devoted to teaching LLM users to write better prompts; for instance, ROPE [49], which trains users to write effective prompts by providing automatic feedback on prompt clarity; Wu et al.'s tool [86], which automatically elaborates based on the LLM's response to the original prompt; and Arawjo et al.'s system [7] for testing and evaluating prompts. In our view, these systems implicitly assume that the main bottleneck is users' ability

to express precise requirements in natural language, which is an assumption that our study lends tentative support for.

Apart from this central finding, we sought to understand how written communication skills impact certain broad types of vibe coding tasks, in particular the difference between non-contextualized tasks and context-rich tasks. Before the study took place, we believed that written communication skills were more important for the former, as the LLM agent could not be relied on to fill in any semantic gaps in students' specifications for completely unfamiliar applications. However, we found no evidence for this hypothesis in our study. Future studies could investigate how other types of vibe coding tasks besides context-rich and decontextualized tasks are impacted by student skills and abilities. Such studies may also investigate broader populations than students and how they perform on different vibe coding tasks.

While our work analyzed how prompt quality and the lexical diversity of participants' prompts relates to and mediates the association between writing proficiency and GUI-oriented vibe-coding performance, future work could categorize prompting styles (e.g., verbose versus curt, iterative workflows versus one-big-prompts) and examine how student skills correlate with such workflows and how it affects their vibe coding performance, although we recommend that any such studies consider explicitly beforehand how the results could inform pedagogical practice or tool design.

Lastly, we find it interesting that prior LLM usage is negatively and significantly correlated with written communication and vibe coding performance. We speculate that this relationship is caused either by (i) LLMs having a negative effect on students' ability to express themselves or (ii) that students who are less proficient at writing are more likely to use LLMs, or (iii) a combination of both. However, the exploratory analysis did not show evidence for LLM usage correlating either negatively nor positively with CS achievement. Other studies have also been ambivalent on the effects of AI exposure on CS and computer programming outcomes; for instance, Kazemitabaar et al.'s controlled study from 2023 [38] found no statistically significant difference in performance on programming retention tests between learners who had access to AI code generators during training and those who did not.

10.1 Result Generalizability

Our study is constrained by its focus on GUI-oriented vibe coding. To probe how far our findings might extend to more algorithmic forms of vibe coding, we conducted an exploratory analysis of the most computation-focused component of our task suite. In the meal-planning task, participants were given a catalog of meals annotated with attributes such as cost, calories, protein, and sugar, and asked to implement behavior that assembled a weekend meal plan under budget and nutritional constraints. This required specifying logic that iterated over the meal dataset, aggregated numeric fields (for example, total and average calories and protein), and compared those aggregates against target thresholds to compute summary statistics and whether they met the given threshold. It is plausible that the performance on this component is an indicator of a participant's ability to compute a simple algorithm via natural-language prompts, as opposed to focusing solely on surface-level GUI actions.

We found that CS achievement and writing skills were also significant predictors of performance on this data-centric aspect of the meal-planning task. CS achievement showed a Pearson correlation of $r = 0.320$ ($p = 0.001$), and writing skills showed a Pearson correlation of $r = 0.202$ ($p = 0.044$). These effect sizes are comparable to, though slightly smaller than, the associations we observed for overall vibe-coding performance. Taken together, they provide tentative evidence that the pattern we observe is not limited to only surface-level GUI construction: both CS knowledge and written communication proficiency also appear to matter when vibe coding requires participants to elicit nontrivial algorithmic behavior from the model. At the same time, this analysis is based on a single task in a specific domain, and should therefore be interpreted cautiously. A fuller assessment of generalizability will require future work that systematically varies task type and includes a broader range of data-centric and algorithm-focused vibe-coding scenarios.

Our results are also constrained by our focus on "no-code" or "pure" vibe coding. In our setting, participants could not inspect or edit the underlying source, so CS knowledge could only support performance indirectly (e.g., through problem decomposition or mental models of control flow and state), rather than through direct code manipulation. As such, our estimates for the role of CS achievement in general LLM-augmented programming should be viewed as a lower bound: in more permissive workflows, the same underlying skills can contribute both via prompt quality and via targeted edits to generated code. At the same time, introducing code access is unlikely to simply scale up the role of CS and scale down the role of writing in a linear way. When both channels are available, users must choose how to distribute work between natural-language specification and code editing; this opens up new strategies (e.g., using prompts for high-level restructuring and code edits for local fixes) and may produce interaction effects between skills. Future work should examine how these skills shape strategy choice and outcomes in mixed-mode environments where reading and editing code are also on the table.

10.2 Limitations and Threats to Validity

Our study has several limitations and threats to validity. First, we note that the vibe coding construct can cover a broader category of tasks than those we studied, for example data analysis. It is possible that our results would be different if we had examined such tasks. Second, our measure of written communication skills, while developed and evaluated by five experts (four of whom worked independently), has not been previously deployed. This might impact its construct validity.

The ecological validity of the study is impacted by the controlled laboratory environment we used, which differs from most real-world vibe-coding settings. In the study, we analyzed "no-code" or "pure" vibe coding only, meaning that participants could not review the code generated, and the results may not translate directly to more permissive LLM-augmented programming workflows. Similarly, all tests and tasks were strictly timed, and we observed in the vibe coding component that several participants ran out of time when they were close to arriving at a correct solution. Task framing may have also affected participants' natural strategies. It is possible that the results were also impacted by students misunderstanding

the tasks they were meant to implement, although task misreading and comprehension skills is likely an unavoidable source of construct-irrelevant variance in this context.

Lastly, we note that our sample consisted of university students. This limits the generalizability of our findings to other populations, such as professional developers or citizen programmers, who may have different vibe-coding workflows than our participants.

11 Conclusion

We presented a study on the impact of measured computer-science achievement, domain-general cognitive ability and written communication skills on the ability to create GUI-oriented applications using a vibe coding approach in which the code is hidden from view. We found that both computer science achievement and written communication skills were positively and significantly correlated with vibe coding skills ($r = .39$ and $r = .29$, respectively), a correlation that persists in the former case after controlling for cognitive ability. Furthermore, both constructs contribute independently to the GUI-oriented vibe coding performance. Our results encourage investigation into the hypothesis that computer science instruction can lead to better outcomes on LLM-guided programming, which is a question that should be investigated in future controlled studies.

Acknowledgments

We thank Dr. Kimberly Lewis and David Camorani from the Language Center of UZH and ETH Zurich for the essay grading rubric. We also thank Kimberly and Dr. Giorgio Iemmolo for the help with the prompt analysis in our exploratory analysis. Any flaws in our work, or in the way that we incorporated their advice, remain our own. We thank the anonymous members of our expert panel and the Decision Science Laboratory at ETH Zurich, and extend special thanks to Dr. Haukur Thorgeirsson, Sandra Wiklander, Anita Barr, and Dr. Tracy Ewen.

References

[1] 2018. ISO/IEC/IEEE International Standard - Systems and software engineering – Life cycle processes – Requirements engineering. *ISO/IEC/IEEE 29148:2018(E)* (2018), 1–104. doi:10.1109/IEEESTD.2018.8559686

[2] ACT, Inc. 2011. *The Condition of College & Career Readiness, 2011.* Technical Report. ACT, Inc. https://files.eric.ed.gov/fulltext/ED522992.pdf Accessed: 2025-06-01.

[3] ACT, Inc. 2016. *The Condition of College & Career Readiness, 2016.* Technical Report. ACT, Inc. https://www.act.org/content/dam/act/unsecured/documents/CCCR_National_2016.pdf Accessed: 2025-06-01.

[4] ACT, Inc. 2020. *ACT Profile Report – National: Graduating Class 2020.* Technical Report. ACT, Inc. https://www.act.org/content/dam/act/unsecured/documents/2020/2020-National-ACT-Profile-Report.pdf Accessed: 2025-06-01.

[5] ACT, Inc. 2024. *2024 ACT National Graduating Class Profile Report.* Technical Report. ACT, Inc. https://www.act.org/content/dam/act/unsecured/documents/2024-act-national-graduating-class-profile-report.pdf Accessed: 2025-06-01.

[6] Hend Alrasheed, Amjad Alnashwan, and Ruwayda Alshowiman. 2021. Impact of English Proficiency on Academic Performance of Software Engineering Students. In *Proceedings of the 2021 4th International Conference on Data Storage and Data Engineering* (Barcelona, Spain) *(DSDE '21)*. Association for Computing Machinery, New York, NY, USA, 107–111. doi:10.1145/3456146.3456163

[7] Ian Arawjo, Chelse Swoopes, Priyan Vaithilingam, Martin Wattenberg, and Elena L Glassman. 2024. Chainforge: A visual toolkit for prompt engineering and LLM hypothesis testing. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–18.

[8] American Educational Research Association, American Psychological Association, National Council on Measurement in Education, et al. 1985. Standards for educational and psychological testing. *(No Title)* (1985).

[9] Association of American Colleges & Universities. 2009. Written Communication VALUE Rubric. https://www.aacu.org/initiatives/value-initiative/value-rubrics/value-rubrics-written-communication VALUE (Valid Assessment of Learning in Undergraduate Education).

[10] John Aycock. 2015. Applied computer history: Experience teaching systems topics through retrogames. In *Proceedings of the 2015 ACM Conference on Innovation and Technology in Computer Science Education*. 105–110.

[11] Shraddha Barke, Michael B. James, and Nadia Polikarpova. 2023. Grounded Copilot: How Programmers Interact with Code-Generating Models. *Proceedings of the ACM on Programming Languages* 7, OOPSLA1, Article 78 (April 2023), 27 pages. doi:10.1145/3586030

[12] Junkai Chen, Li Zhenhao, Hu Xing, and Xia Xin. 2025. NLPerturbator: Studying the Robustness of Code LLMs to Natural Language Variations. *ACM Trans. Softw. Eng. Methodol.* (July 2025). doi:10.1145/3745764

[13] Eugene Cho. 2019. Hey Google, Can I Ask You Something in Private?. In *Proceedings of the 2019 CHI Conference on Human Factors in Computing Systems* (Glasgow, Scotland Uk) *(CHI '19)*. Association for Computing Machinery, New York, NY, USA, 1–9. doi:10.1145/3290605.3300488

[14] Dakota W Cintron. 2021. Methods for measuring speededness: Chronology, classification, and ensuing research and development. *ETS Research Report Series* 2021, 1 (2021), 1–36.

[15] Collins English Dictionary. 2025. "vibe coding". *Collins English Dictionary* (online). https://www.collinsdictionary.com/dictionary/english/vibe-coding Accessed 27 November 2025.

[16] David M Condon and William Revelle. 2014. The international cognitive ability resource: Development and initial validation of a public-domain measure. *Intelligence* 43 (2014), 52–64.

[17] Cursor. 2025. *Cursor – The AI Code Editor.* https://cursor.com/home Accessed on 2025-08-27.

[18] Cursor Documentation. 2025. *VS Code Migration Guide.* https://docs.cursor.com/en/guides/migration/vscode Accessed on 2025-08-27.

[19] Edsger W Dijkstra. 2005. On the foolishness of "natural language programming". *Program Construction: International Summer School* (2005), 51–53.

[20] Elizabeth M Dworak, William Revelle, Philip Doebler, and David M Condon. 2021. Using the International Cognitive Ability Resource as an open source tool to explore individual differences in cognitive ability. *Personality and Individual Differences* 169 (2021), 109906.

[21] Li Feng, Ryan Yen, Yuzhe You, Mingming Fan, Jian Zhao, and Zhicong Lu. 2024. Coprompt: Supporting prompt sharing and referring in collaborative natural language programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. doi:10.1145/3613904.3642212

[22] National Center for Education Statistics. 2025. *2024 NAEP Reading Assessment: Results at Grades 4 and 8 for the Nation, States, and Districts.* Technical Report NCES 2024218. U.S. Department of Education. https://nces.ed.gov/use-work/resource-library/report/statistical-analysis-report/2024-naep-reading-assessment-results-grades-4-and-8-nation-states-and-districts Accessed: 2025-06-01.

[23] Francis Geng, Anshul Shah, Haolin Li, Nawab Mulla, Steven Swanson, Gerald Soosai Raj, Daniel Zingaro, and Leo Porter. 2025. Exploring Student-AI Interactions in Vibe Coding. *arXiv preprint arXiv:2507.22614* (2025).

[24] Google Cloud. 2025. Replit: From idea to application in minutes. https://cloud.google.com/customers/replit Archived at https://web.archive.org/web/20250729010057/https://cloud.google.com/customers/replit (Wayback Machine, captured 2025-07-29)..

[25] Irene L. Graafsma, Serje Robidoux, Lyndsey Nickels, Matthew Roberts, Vince Polito, Judy D. Zhu, and Eva Marinus. 2023. The cognition of programming: logical reasoning, algebra and vocabulary skills predict programming performance following an introductory computing course. *Journal of Cognitive Psychology* 35, 3 (2023), 364–381. doi:10.1080/20445911.2023.2166054

[26] Jr. Hair, Joseph F., William C. Black, Barry J. Babin, and Rolph E. Anderson. 2019. *Multivariate Data Analysis* (8th ed.). Cengage Learning EMEA, Andover, Hampshire.

[27] Kevin A Hallgren. 2012. Computing inter-rater reliability for observational data: an overview and tutorial. *Tutorials in quantitative methods for psychology* 8, 1 (2012), 23.

[28] Mark Halpern. 1966. Foundations of the case for natural-language programming. In *Proceedings of the November 7-10, 1966, Fall Joint Computer Conference*. 639–649. doi:10.1145/1464291.1464360

[29] Andrew F. Hayes. 2022. *Introduction to Mediation, Moderation, and Conditional Process Analysis: A Regression-Based Approach* (3rd ed.). The Guilford Press, New York, NY.

[30] Michael A Hebert and Sarah R Powell. 2016. Examining fourth-grade mathematics writing: Features of organization, mathematics vocabulary, and mathematical representations. *Reading and Writing* 29, 7 (2016), 1511–1537.

[31] Anna Heim. 2025. Eight months in, Swedish unicorn Lovable crosses the $100M ARR milestone. *TechCrunch* (2025). https://techcrunch.com/2025/07/23/eight-months-in-swedish-unicorn-lovable-crosses-the-100m-arr-milestone/

[32] International Cognitive Ability Resource Team. 2021. *International Cognitive Ability Resource Guidelines.* International Cognitive Ability Resource (ICAR). https://icar-project.com/attachments/download/137/Guidelines.pdf

[33] Anna A Ivanova, Shashank Srikant, Yotaro Sueoka, Hope H Kean, Riva Dhamala, Una-May O'reilly, Marina U Bers, and Evelina Fedorenko. 2020. Comprehension of computer code relies primarily on domain-general executive brain regions. *elife* 9 (2020), e58906.

[34] Johan Jeuring, Roel Groot, and Hieke Keuning. 2024. What Skills Do You Need When Developing Software Using ChatGPT? (Discussion Paper). In *Proceedings of the 23rd Koli Calling International Conference on Computing Education Research* (Koli, Finland) *(Koli Calling '23)*. Association for Computing Machinery, New York, NY, USA, Article 38, 6 pages. doi:10.1145/3631802.3631807

[35] Ellen Jiang, Edwin Toh, Alejandra Molina, Kristen Olson, Claire Kayacik, Aaron Donsbach, Carrie J Cai, and Michael Terry. 2022. Discovering the Syntax and Strategies of Natural Language Programming with Generative Language Models. In *Proceedings of the 2022 CHI Conference on Human Factors in Computing Systems* (New Orleans, LA, USA) *(CHI '22)*. Association for Computing Machinery, New York, NY, USA, Article 386, 19 pages. doi:10.1145/3491102.3501870

[36] Ulas Berk Karli, Juo-Tung Chen, Victor Nikhil Antony, and Chien-Ming Huang. 2024. Alchemist: LLM-Aided End-User Development of Robot Applications. In *Proceedings of the 2024 ACM/IEEE International Conference on Human-Robot Interaction* (Boulder, CO, USA) *(HRI '24)*. Association for Computing Machinery, New York, NY, USA, 361–370. doi:10.1145/3610977.3634969

[37] Andrej Karpathy. 2025. There's a new kind of coding I call 'vibe coding'... https://x.com/karpathy/status/1886192184808149383. Tweet on X (formerly Twitter). Posted 2025-02-02 23:17 UTC. Accessed 2025-08-16.

[38] Majeed Kazemitabaar, Justin Chow, Carl Ka To Ma, Barbara J Ericson, David Weintrop, and Tovi Grossman. 2023. Studying the effect of AI code generators on supporting novice learners in introductory programming. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. 1–23. doi:10.1145/3544548.3580919

[39] Kristof Keidel, Xiaping Lu, Shinsuke Suzuki, Carsten Murawski, and Ulrich Ettinger. 2024. Association of temporal discounting with transdiagnostic symptom dimensions. *npj Mental Health Research* 3, 1 (2024), 13. doi:10.1038/s44184-024-00060-3

[40] Ivan Klimek, Marián Keltika, and František Jakab. 2011. Reverse engineering as an education tool in computer science. In *2011 9th International Conference on Emerging eLearning Technologies and Applications (ICETA)*. 123–126. doi:10.1109/ICETA.2011.6112599

[41] D. E. Knuth. 1984. Literate Programming. *Comput. J.* 27, 2 (01 1984), 97–111. doi:10.1093/comjnl/27.2.97

[42] Tobias Kuhn. 2014. A survey and classification of controlled natural languages. *Computational linguistics* 40, 1 (2014), 121–170.

[43] Sam Lau and Philip J. Guo. 2025. The Design Space of LLM-Based AI Coding Assistants: An Analysis of 90 Systems in Academia and Industry. In *2025 IEEE Symposium on Visual Languages and Human-Centric Computing (VL/HCC)*. IEEE, Raleigh, NC, USA. https://lau.ucsd.edu/pubs/2025_analysis-of-90-genai-coding-tools_VLHCC.pdf Preprint; proceedings not published..

[44] Robert R Leeper and James L Silver. 1982. Predicting success in a first programming course. *ACM SIGCSE Bulletin* 14, 1 (1982), 147–150.

[45] Harold A Linstone, Murray Turoff, et al. 1975. *The Delphi method*. Vol. 1975. Addison-Wesley Reading, MA.

[46] Michael Xieyang Liu, Advait Sarkar, Carina Negreanu, Benjamin Zorn, Jack Williams, Neil Toronto, and Andrew D. Gordon. 2023. "What It Wants Me To Say": Bridging the Abstraction Gap Between End-User Programmers and Code-Generating Large Language Models. In *Proceedings of the 2023 CHI Conference on Human Factors in Computing Systems* (Hamburg, Germany) *(CHI '23)*. Association for Computing Machinery, New York, NY, USA, Article 598, 31 pages. doi:10.1145/3544548.3580817

[47] Yun-Fei Liu, Judy Kim, Colin Wilson, and Marina Bedny. 2020. Computer code comprehension shares neural resources with formal logical inference in the fronto-parietal network. *Elife* 9 (2020), e59340.

[48] Francesca Lucchetti, Zixuan Wu, Arjun Guha, Molly Feldman, and Carolyn Anderson. 2025. Substance Beats Style: Why Beginning Students Fail to Code with LLMs. In *Proceedings of the 2025 Conference of the Nations of the Americas Chapter of the Association for Computational Linguistics: Human Language Technologies*. 8541–8610. doi:10.18653/v1/2025.naacl-long.433

[49] Qianou Ma, Weirui Peng, Chenyang Yang, Hua Shen, Ken Koedinger, and Tongshuang Wu. 2025. What Should We Engineer in Prompts? Training Humans in Requirement-Driven LLM Use. *ACM Trans. Comput.-Hum. Interact.* 32, 4, Article 41 (Aug. 2025), 27 pages. doi:10.1145/3731756

[50] Amjad Masad. 2025. *Vibe coding is already here. 75% of Replit customers never write a single line of code!!* X (formerly Twitter). https://x.com/amasad/status/1886516600653930924 Post on X.

[51] Damien Masson, Sylvain Malacria, Géry Casiez, and Daniel Vogel. 2024. DirectGPT: A direct manipulation interface to interact with large language models. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems*. 1–16.

[52] Philip M McCarthy and Scott Jarvis. 2010. MTLD, vocd-D, and HD-D: A validation study of sophisticated approaches to lexical diversity assessment. *Behavior research methods* 42, 2 (2010), 381–392.

[53] Kathryne Drezek McConnell, Erin M Horan, Bethany Zimmerman, and Terrel L Rhodes. 2019. *We Have a Rubric for That: The VALUE Approach to Assessment*. ERIC.

[54] Daniel J McNair and Toi L Curry. 2013. The Forgotten: Formal Assessment of the Adult Writer. *Journal of Postsecondary Education and Disability* 26, 1 (2013), 5–19.

[55] Rachel Metz. 2025. AI Coding Assistant Cursor Draws a Million Users Without Even Trying. *Bloomberg* (2025). https://www.bloomberg.com/news/articles/2025-04-07/cursor-an-ai-coding-assistant-draws-a-million-users-without-even-trying

[56] Emmanuel OC Mkpojiogu and Azham Hussain. 2017. Can scoring rubrics be used in assessing the performance of students in software requirements engineering education? *Journal of Telecommunication, Electronic and Computer Engineering (JTEC)* 9, 2-11 (2017), 115–119.

[57] Hussein Mozannar, Gagan Bansal, Adam Fourney, and Eric Horvitz. 2024. Reading Between the Lines: Modeling User Behavior and Costs in AI-Assisted Programming. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 142, 16 pages. doi:10.1145/3613904.3641936

[58] National Assessment Governing Board. 2010. *Writing Framework for the 2011 National Assessment of Educational Progress*. Technical Report. National Assessment Governing Board, U.S. Department of Education, Washington, DC. 108 pages. https://www.nagb.gov/content/dam/nagb/en/documents/publications/frameworks/writing/2011-writing-framework.pdf

[59] National Center for Education Statistics. 2024. Explore Results for the 2024 NAEP Reading Assessment. The Nation's Report Card, U.S. Department of Education. https://www.nationsreportcard.gov/reports/reading/2024/g4_8/ Accessed: 2025-06-01.

[60] Sydney Nguyen, Hannah McLean Babe, Yangtian Zi, Arjun Guha, Carolyn Jane Anderson, and Molly Q Feldman. 2024. How Beginning Programmers and Code LLMs (Mis)read Each Other. In *Proceedings of the 2024 CHI Conference on Human Factors in Computing Systems* (Honolulu, HI, USA) *(CHI '24)*. Association for Computing Machinery, New York, NY, USA, Article 651, 26 pages. doi:10.1145/3613904.3642706

[61] Jum C. Nunnally and Ira H. Bernstein. 1994. *Psychometric Theory* (3rd ed.). McGraw-Hill, New York.

[62] OECD. 2023. *PISA 2022 Results (Volume I): The State of Learning and Equity in Education*. OECD Publishing, Paris. doi:10.1787/53f23881-en

[63] Association of American Colleges and Universities. [n. d.]. *VALUE Rubrics*. AAC and U. https://www.aacu.org/value/rubrics

[64] Council of Europe. Council for Cultural Co-operation. Education Committee. Modern Languages Division. 2001. *Common European framework of reference for languages: Learning, teaching, assessment*. Cambridge University Press.

[65] Organisation for Economic Co-operation and Development (OECD). 2024. *Survey of Adult Skills 2023: United States*. Technical Report. OECD Publishing. https://www.oecd.org/en/publications/survey-of-adults-skills-2023-country-notes_ab4f6b8c-en/united-states_427d6aac-en.html Accessed: 2025-06-01.

[66] Matt Palmer. 2025. *What is Vibe Coding? How To Vibe Your App to Life*. Replit. https://blog.replit.com/what-is-vibe-coding Blog post; updated 2025-07-08.

[67] Miranda C Parker, Mark Guzdial, and Shelly Engleman. 2016. Replication, validation, and use of a language independent CS1 knowledge assessment. In *Proceedings of the 2016 ACM Conference on International Computing Education Research*. 93–101. doi:10.1145/2960310.2960316

[68] Miranda C Parker, Mark Guzdial, and Allison Elliott Tew. 2021. Uses, revisions, and the future of validated assessments in computing education: A case study of the FCS1 and SCS1. In *Proceedings of the 17th ACM Conference on International Computing Education Research*. 60–68.

[69] Miranda C Parker, Amber Solomon, Brianna Pritchett, David A Illingworth, Lauren E Marguilieux, and Mark Guzdial. 2018. Socioeconomic status and computer science achievement: Spatial ability as a mediating variable in a novel model of understanding. In *Proceedings of the 2018 ACM Conference on International Computing Education Research*. 97–105. doi:10.1145/3230977.3230987

[70] Madison Pickering, Helena Williams, Alison Gan, Weijia He, Hyojae Park, Francisco Piedrahita Velez, Michael L. Littman, and Blase Ur. 2025. How Humans Communicate Programming Tasks in Natural Language and Implications For End-User Programming with LLMs. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems (CHI '25)*. Association for Computing Machinery, New York, NY, USA, Article 875, 34 pages. doi:10.1145/3706598.3713271

[71] Chantel S Prat, Tara M Madhyastha, Malayka J Mottarella, and Chu-Hsuan Kuo. 2020. Relating natural language aptitude to individual differences in learning programming languages. *Scientific reports* 10, 1 (2020), 3817.

[72] Replit. 2025. *Replit Closes $250 Million in Funding to Build on Customer Momentum*. Replit. https://replit.com/news/funding-announcement Archived at https://web.archive.org/web/20260122/https://replit.com/news/funding-announcement.

[73] Kevin Roose. 2025. Not a Coder? With A.I., Just Having an Idea Can Be Enough. *The New York Times* (2025). https://www.nytimes.com/2025/02/27/technology/personaltech/vibecoding-ai-software-programming.html Personal Tech section.

[74] Jean E Sammet. 1961. Detailed description of COBOL. In *International Tracts in Computer Science and Technology and Their Application*. Vol. 2. Elsevier, 197–230.

[75] Advait Sarkar and Ian Drosos. 2025. Vibe coding: programming through conversation with artificial intelligence. *arXiv preprint arXiv:2506.23253* (2025).

[76] Daniela Schriebl, Andreas Müller, and Nicolas Robin. 2023. Modelling Authenticity in Science Education. *Science & Education* 32, 4 (2023), 1021–1048. doi:10.1007/s11191-022-00355-x

[77] Janet Siegmund, Christian Kästner, Sven Apel, Chris Parnin, Anja Bethmann, Thomas Leich, Gunter Saake, and André Brechmann. 2014. Understanding understanding source code with functional magnetic resonance imaging. In *Proceedings of the 36th international conference on software engineering*. 378–389.

[78] Jasmine Sun. 2025. Your Next Favorite App? The One You Make Yourself. (2025). https://www.wsj.com/tech/ai/your-next-favorite-app-the-one-you-make-yourself-a6a84f5f Published 9:54 a.m. ET; section: Tech/AI.

[79] JP Wahle, T Ruas, Y Xu, and B Gipp. 2023. The language of prompting: what linguistic properties make a prompt successful. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. 1234–1245.

[80] David Wechsler. 2001. Wechsler Individual Achievement Test–Second Edition (WIAT-II) [Database record]. APA PsycTests. doi:10.1037/t15173-000

[81] David Wechsler et al. 1991. *Wechsler intelligence scale for children*. Vol. 2195. Psychological Corporation San Antonio, TX.

[82] Eric Wiebe, Jennifer London, Osman Aksit, Bradford W Mott, Kristy Elizabeth Boyer, and James C Lester. 2019. Development of a lean computational thinking abilities assessment for middle grades students. In *Proceedings of the 50th ACM technical symposium on computer science education*. 456–461. doi:10.1145/3287324.3287390

[83] Simon Willison. 2025. Not all AI-assisted programming is vibe coding (but vibe coding rocks). https://simonwillison.net/2025/Mar/19/vibe-coding/. Simon Willison's Weblog. Accessed: 2025-11-27.

[84] Emily A. Willoughby, Matt McGue, William G. Iacono, and James J. Lee. 2021. Genetic and environmental contributions to IQ in adoptive and biological families with 30-year-old offspring. *Intelligence* 88 (2021), 101579. doi:10.1016/j.intell.2021.101579

[85] Kelly Woods, Brett Hashimoto, and Earl K Brown. 2023. A multi-measure approach for lexical diversity in writing assessments: Considerations in measurement and timing. *Assessing Writing* 55 (2023), 100688.

[86] Tongshuang Wu, Michael Terry, and Carrie Jun Cai. 2022. AI chains: Transparent and controllable human-AI interaction by chaining large language model prompts. In *Proceedings of the 2022 CHI conference on human factors in computing systems*. 1–22.

[87] YJ Xie and Y Shen. 2015. A study of the relationships between lexical richness and writing quality: Taking the English majors at Guangxi University as an example. In *2015 International Conference on Social Science, Education Management and Sports Education*. Atlantis Press, 1638–1641.

[88] Yang Yang, Ngee Thai Yap, and Afida Mohamad Ali. 2023. Predicting EFL expository writing quality with measures of lexical richness. *Assessing Writing* 57 (2023), 100762.

[89] Stephanie R. Young and Timothy Z. Keith. 2020. An Examination of the Convergent Validity of the ICAR16 and WAIS-IV. *Journal of Psychoeducational Assessment* 38, 8 (2020), 1052–1059. doi:10.1177/0734282920943455

[90] Stephanie Ruth Young, Timothy Z. Keith, and Mark A. Bond. 2019. Age and sex invariance of the International Cognitive Ability Resource (ICAR). *Intelligence* 77 (2019), 101399. doi:10.1016/j.intell.2019.101399

[91] Mireia Yurrita, Himanshu Verma, Agathe Balayn, Ujwal Gadiraju, Sylvia C Pont, and Alessandro Bozzon. 2025. Towards Effective Human Intervention in Algorithmic Decision-Making: Understanding the Effect of Decision-Makers' Configuration on Decision-Subjects' Fairness Perceptions. In *Proceedings of the 2025 CHI Conference on Human Factors in Computing Systems*. 1–21. doi:10.1145/3706598.3713145

[92] J Diego Zamfirescu-Pereira, Richmond Y Wong, Bjoern Hartmann, and Qian Yang. 2023. Why Johnny can't prompt: how non-AI experts try (and fail) to design LLM prompts. In *Proceedings of the 2023 CHI conference on human factors in computing systems*. Association for Computing Machinery, New York, NY, USA, 1–21. doi:10.1145/3544548.3581388